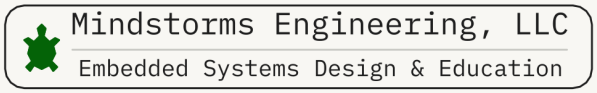
You Don't Need an RTOS (Part 3)

by Nathan Jones



At least, probably not as often as you think you do. Using a preemptive RTOS can help make a system schedulable (i.e. help all of its tasks meet their deadlines) but it comes at a cost: "concurrently" running tasks can interact in unforeseen ways that cause system failures, that dreaded class of errors known as "race conditions". In part, this is because humans have difficulty thinking about things that happen concurrently. Using the venerable superloop can help avoid this problem entirely and, in some cases, it can actually make a system work that wouldn't work at all using an RTOS! Additionally, a "superloop" style scheduler has the benefit of being simple enough for you to write yourself in a few dozen lines of code.

If you're unfamiliar with terms like "deadlines", "schedulability", or "worst-case response time", then I would argue you are almost *definitely* using an RTOS prematurely! If you *are* familiar with those terms, allow me to show you a few ways to improve the superloop that bring its performance nearly on par with an RTOS, with no concern about race conditions.

This is the third in a series of posts lauding the advantages of the simple "superloop" scheduler. Other articles in this series are or will be linked below as they are published.

- Part 1: Comparing Superloops and RTOSes
- Part 2: The "Superduperloop"
- Part 3: Other Cool Superloops and DIY IPC (this article)
- Part 4: More DIY IPC

Contents

Part 3: Other Cool Superloops and DIY IPC

In this third article I'll share with you the following cooperative schedulers (with a mix of both free and commercial licenses) that implement a few of the OS primitives that the "Superduperloop" is currently missing, possibly giving you a ready-to-go solution for your system:

- Protothreads
- ULWOS2 (Ultra Light Weight OS 2)
- cocoOS
- SST0 (Super Simple Tasker) / QV (from the QP Framework)

On the other hand, I don't think it's all that hard to add thread flags, binary and counting semaphores, event flags, mailboxes, "marquees", queues, and even a simple Observer design pattern to the "Superduperloop" (you can find lots of great information about what each of these primitives do by reading the documentation for an RTOS like FreeRTOS or CMSIS RTX5); I'll show you how to do that in the second half of this article and in the next. Although it will take a *little* more work than just using one of the projects above, it will give you the *maximum* amount of control over your system and it will let you write tasks in ways you could only dream of using an RTOS or other off-the-shelf system.

A Quick Recap

In Part 2 of this series we added task priorities and interrupts to the basic superloop (and looked at how to turn tasks into FSMs [finite state machines]) to give it a WCRT (worst-case response time) on par with a preemptive RTOS (real-time operating system). That scheduler, the "Superduperloop", looked like this:

```
// One bitmask per task, e.g.
enum { A_NUM, B_NUM, NUM_TASKS };
const int A_MASK = (1<<A_NUM);
const int B_MASK = (1<<B_NUM);
// Function prototypes for tasks here, e.g.
int TaskA(void);
int TaskB(void);
volatile int g_systick = 0, g_tasks = 0;
void timerISR(void)
```

```
{
    g_systick++;
    // Release a task using, e.g.
    // atomic_fetch_or(&g_tasks, A_MASK);  // Assuming ISRs are preemptible.
Otherwise
                                           // "g_tasks |= A_MASK;" is fine.
}
void main(void)
{
    while(1)
    {
        // One if/else if block per task, higher priority tasks located
        // higher up in the chain of if statements.
        //
        if( g_tasks & A_MASK )
        {
            atomic_fetch_and(&g_tasks, (~A_Mask));
            if( TaskA() ) atomic_fetch_or(&g_tasks, A_MASK);
        }
        else if( g_tasks & B_MASK )
        {
            atomic_fetch_and(&g_tasks, (~B_Mask));
            if( TaskB() ) atomic_fetch_or(&g_tasks, B_MASK);
        }
        else
        {
            if( !g_tasks )
            {
                disableInterrupts();
                if( !g_tasks ) wfi();
                enableInterrupts();
            }
        }
    }
}
```

The WCRT of a task was found using equation 5 (or equation 3 or 4 for an ISR) and task priorities were assigned using either DMS (deadline monotonic scheduling), an exhaustive search, or Audsley's algorithm (see Assigning Task Priorities).

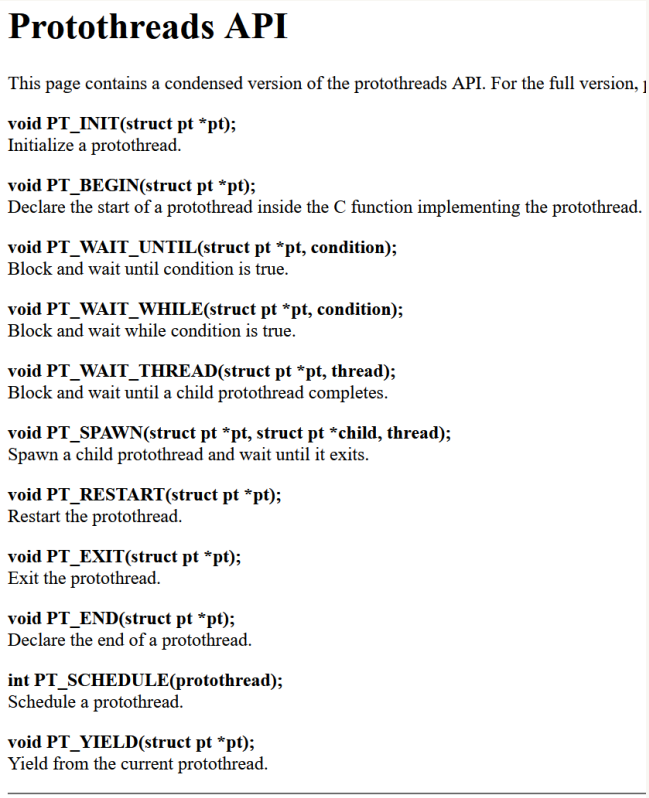
Other Cool Superloops

At the end of that article, though, we established that for this to be a viable solution for an embedded system, our scheduler needs to support a few forms of **inter-process communication** (IPC). These forms of communication or synchronization are what allow tasks to interact in a correct manner. Typical forms of IPC are semaphores, message queues, mutexes,

stream buffers, thread flags, and event flags. In the second half of this article and in the next I'll show you how to implement those OS primitives in the "Superduperloop". But if you're ready to see how someone else has already tried to solve that problem, here are a handful of cooperative schedulers that you can use right now in your next embedded project.

Protothreads

Protothreads isn't a scheduler, per se, but we've used it a bit in the last two articles to show how it can be used to greatly simplify your FSM-based tasks. In those examples you saw a basic use of Protothreads, but this project has some other neat function calls to assist in writing your application code that include waiting *while* a condition is true (as opposed to *until* a condition is true), spawning threads, and waiting for a thread to complete, among others.



Protothreads API

This page contains a condensed version of the protothreads API. For the full version,

void PT_INIT(struct pt *pt);
Initialize a protothread.

void PT_BEGIN(struct pt *pt);
Declare the start of a protothread inside the C function implementing the protothread.

void PT_WAIT_UNTIL(struct pt *pt, condition);
Block and wait until condition is true.

void PT_WAIT_WHILE(struct pt *pt, condition);
Block and wait while condition is true.

void PT_WAIT_THREAD(struct pt *pt, thread);
Block and wait until a child protothread completes.

void PT_SPAWN(struct pt *pt, struct pt *child, thread);
Spawn a child protothread and wait until it exits.

void PT_RESTART(struct pt *pt);
Restart the protothread.

void PT_EXIT(struct pt *pt);
Exit the protothread.

void PT_END(struct pt *pt);
Declare the end of a protothread.

int PT_SCHEDULE(protothread);
Schedule a protothread.

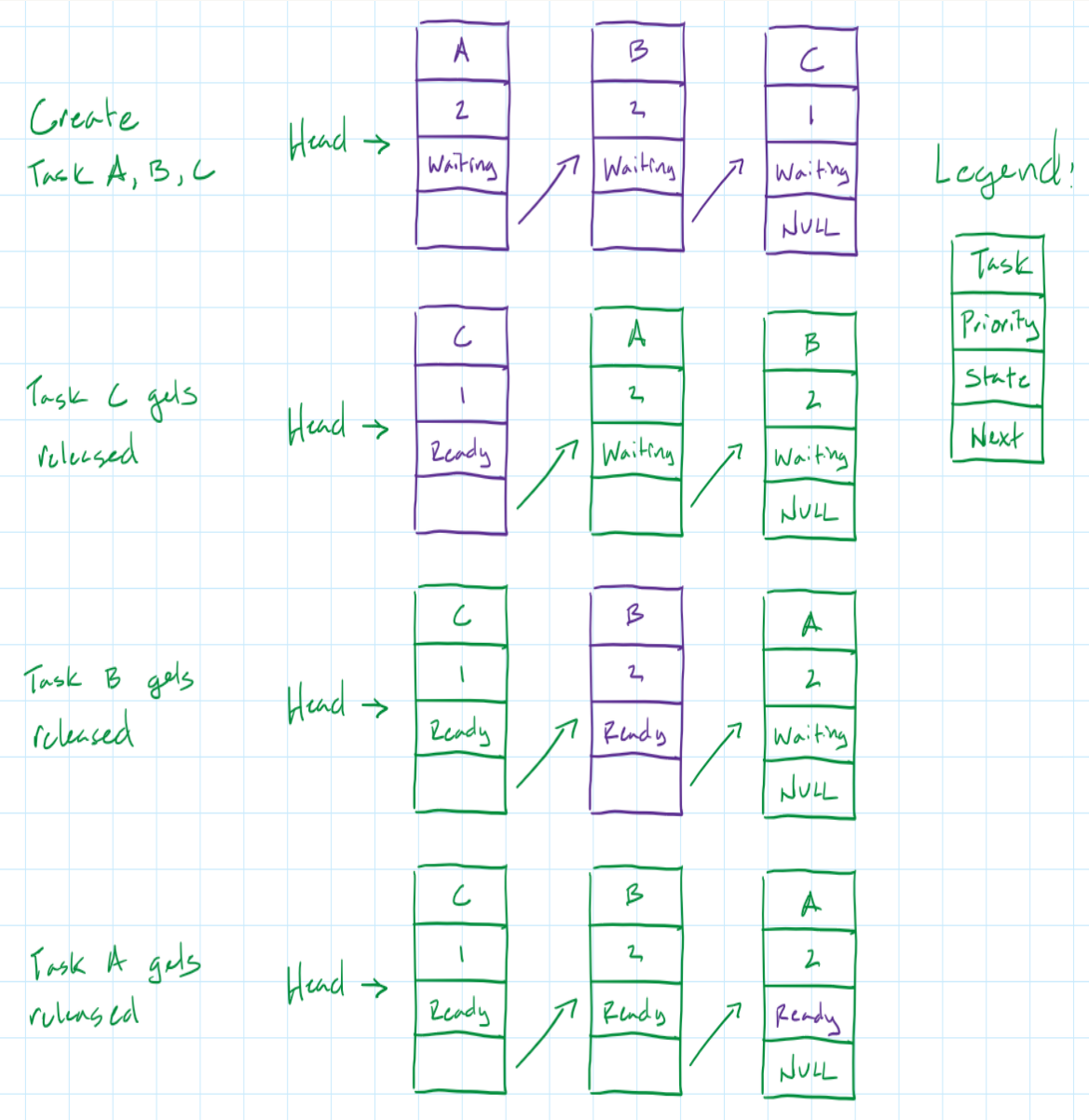
void PT_YIELD(struct pt *pt);
Yield from the current protothread.

The author also includes a basic implementation for a semaphore, though I think you'll need to make sure that interrupts are off when you use those functions, to prevent any race conditions from ISRs modifying the semaphore at the same time. Overall, I think it's a great companion for the "Superduperloop" and I'll use it below to demonstrate what a task might look like when using all of the OS primitives that we'll develop in the second half of this article.

ULWOS2 (Ultra Light Weight OS 2)

ULWOS2 is a simple cooperative scheduler in which tasks are added to a linked list as they are created and which then get resorted (and then executed) in priority order any time a task becomes ready to run. One neat feature of this design choice is that an arbitrary number of tasks in ULWOS2 are allowed to have the same priority level. The graphic below shows what this

linked list of tasks might look like and what would happen when these tasks are released in the order shown.



The *downside* of having tasks at the same priority level is that our scheduling math gets more complicated. The ULWOS2 scheduler will only push tasks with a higher priority ahead of tasks with a lower priority; for tasks with the same priority level, the task that runs first is, essentially, the task that was released first. Notice how, in the graphic above, Tasks C and B are moved *up* in the linked list as they become ready to run, but Task A remains in place in the linked list (since it doesn't have a *strictly* higher priority than Task B, which is already ready to run). In effect, this creates something like a **dynamic priority scheme** and the exact WCRT of tasks at the same priority level will change based on the specific release times of those tasks. The best we can do, when using this scheduler, is say that each tasks' WCRT is *no worse* than if it were the *last to run* within its priority level and *no better* than if it were the *first to run*. (We'll discuss dynamic priority schedulers in a future article.) There are no complications if there is only one task at each priority level.

ULWOS2 is based on a set of macros that create `goto` targets within a task for the same reason that Protothreads used `switch...case` statements, allowing a task to pick back up in the same spot that it yielded from on subsequent function calls.

```
1.  #define ULWOS2_THREAD_YIELD() ({jumper= &&GLUE3(LB,__FUNCTION__,__LINE__); return; GLUE3(LB,__FUNCTION__,__LINE__):
    while(0);})
```

ULWOS2 supports two forms of signaling: event flags (which are called "signals") and message queues. Adding support for other OS primitives (such as semaphores or thread flags) would take a little bit more work, but I think it would not be impossible.

- **ULWOS2_INIT** – initializes ULWOS2 (should be called before creating any thread);
- **ULWOS2_THREAD_CREATE** – creates a new thread with a specific priority;
- **ULWOS2_THREAD_KILL** – to kill (destroy) a thread;
- **ULWOS2_START_SCHEDULER** – start ULWOS2 scheduler
- **ULWOS2_THREAD_START** – this is the constructor that enables jumping to a specific point within the thread when resuming;
- **ULWOS2_THREAD_YIELD** – yields back to the scheduler. This is used for giving a chance of other threads to run (it doesn't block the thread);
- **ULWOS2_THREAD_SLEEP_MS** – blocks the thread for a specific amount of milliseconds;
- **ULWOS2_THREAD_SEND_SIGNAL** – sends a signal to other threads;
- **ULWOS2_THREAD_WAIT_FOR_SIGNAL** – suspends the thread until the given signal arrives;
- **ULWOS2_THREAD_RESET** – reinitialize the thread internal pointer (but does not touch any local variable). This forces the thread to run from the beginning;

The ULWOS2 queue API includes (among a few others), the functions below. One big downside to ULWOS2, though, is that **none of these queue functions are thread-safe**, meaning that you'd need to disable interrupts before (and re-enable them after) each function call if there's any chance of that piece of code being interrupted by an ISR that could *also* operate on that same queue (or you could fork the project and modify it to use atomic functions so that accesses to each queue *were* thread-safe).

- `ULWOS2_MESSAGE_ENQUEUE(id,data)`
- `ULWOS2_MESSAGE_TRY_ENQUEUE(id,data)`
- `ULWOS2_MESSAGE_DEQUEUE(id,data)`
- `ULWOS2_MESSAGE_TRY_DEQUEUE(id,data)`

Putting an MCU to sleep using ULWOS2 is a little problematic; the scheduler is set up to just spin forever, looking for tasks that are ready to run. The best option would probably be to put a `wfi()` call inside a task at the lowest priority level (since no other tasks would need to run if the scheduler got there), but you'd leave yourself open to that once-in-a-blue-moon error when, right after the ULWOS2 scheduler decided to run the lowest priority task (and go to sleep), an interrupt *did* run, releasing a higher priority task. That task would then have to wait for the *next* interrupt to occur to wake-up the processor and restart the scheduler. (If you're forking the project to fix the thread-safety of the queue functions, you might consider adding the following

code snippet here [right after the end of the `while` loop and before `ULWOS2_checkTimers()`] to let your system go to sleep safely):

```
if( currentTCB == NULL )
{
    disableInterrupts();
    if( !invalidateThreadPriorityQueue ) wfi();
    enableInterrupts();
}
```

Here's what a simple ULWOS2 application looks like.

```
#include <ULWOS2.h>

/*
  ULWOS2 - single thread running LED blinker on Arduino
  Author: Fábio Pereira
  Please visit www.embeddedsystems.io for more information on ULWOS2
*/

// This is our thread, it blinks the builtin LED without blocking code
void thread1(void)
{
  ULWOS2_THREAD_START();        // this is necessary for each thread in ULWOS2!
  while(1)
  {
    digitalWrite(LED_BUILTIN, HIGH);    // turn LED on
    ULWOS2_THREAD_SLEEP_MS(50);       // sleep for 50ms
    digitalWrite(LED_BUILTIN, LOW);       // turn LED off
    ULWOS2_THREAD_SLEEP_MS(500);            // sleep for 500ms
  }
}

void setup() {
  ULWOS2_INIT();        // initialize ULWOS2
  ULWOS2_THREAD_CREATE(thread1, 10);    // create thread 1 with priority 10
  pinMode(LED_BUILTIN, OUTPUT); // initialize LED I/O pin as output
}

void loop() {
  // run ULWOS2 scheduler (it will run all threads that are ready to run)
  ULWOS2_START_SCHEDULER();
}
```

To me, the best part about ULWOS2 is that it resides in only 4 source files! You can find the source code here.

cocoOS

cocoOS is a larger cooperative scheduler that adds tasks to an array as they are created and then determines which tasks are ready to run anytime an event occurs by iterating over that array. cocoOS uses the `task_open()` and `task_close()` macros at the beginning and end of a task to open and close a `switch...case` statement, much like Protothreads does.

```
#define task_open() OS_BEGIN
#define OS_BEGIN    uint16_t os_task_state =
os_task_internal_state_get(running_tid); \
                    switch ( os_task_state ) { case 0:
```

cocoOS supports a number of different IPC, including events, semaphores, and message queues. Additionally, cocoOS provides an interesting form of IPC based on "subticks", which are counters that can be incremented by events which are not merely a timer ISR. Tasks can wait until a number of "subticks" have transpired (much like waiting until a number of clock ticks have transpired), which is the same as waiting until some number of events that trigger the subtick have occurred. Below is a summary of some of the most important API calls.

```
- task_wait()
- event_wait()
- event_wait_timeout()
- event_wait_multiple()
- event_signal()
- sem_wait()
- sem_signal()
- msg_post()
- msg_post_in()
- msg_post_every()
- msg_receive()
```

cocoOS is the largest piece of code on this list (weighing in at over *2000 LOC*), and a big reason for that length is that the API goes above and beyond what you might expect from a simple implementation of events, semaphores, and queues.

- Tasks can wait for events (`event_wait`) and also wait for events *or* until a timeout is reached (`event_wait_timeout`).
- Tasks can wait for an arbitrary number of events to occur (`event_wait_multiple`).
- Tasks can post messages to another task's queue and, if the queue is full, either yield until the queue is not full (`msg_post`) or abandon the posting of the message and continue execution (`msg_post_async`). Similarly, tasks can try to pull a message out of their queue and, if the queue is empty, either yield until the queue is not empty (`msg_receive`) or abandon trying to receive a message and continue execution (`msg_receive_async`).

Some of the features, though, don't seem all that useful to me.

- Tasks can specify that a message not post to another task's queue until after a delay (`msg_post_in`) or that a message be posted to a task's queue at regular intervals (`msg_post_every`).

- Tasks can call various functions with a callback parameter that calls a function once the task's state has been updated (`event_wait_ex`, `event_wait_timeout_ex`, `msg_receive_ex`).
- Tasks can suspend themselves or another task (`task_suspend`), in which case they don't run or respond to events until they are resumed (`task_resume`).

There are a few other downsides to this project. First, **none of the code (that I can tell) is thread-safe**. And this, *despite offering two functions called `event_ISR_signal` and `sem_ISR_signal`*!! (Actually, it turns out those two functions aren't called that because they're ISR-safe, but because they're safe to be called from within an ISR; they do the same things as `event_signal` and `sem_signal` except they don't yield control back to the scheduler when they run.) Near as I can tell, all of the functions that modify a task's state fail to protect the critical sections where shared memory is accessed, which leaves the program open to race conditions.

Lastly, although cocoOS provides the system the ability to sleep if there are no pending tasks, ISRs are not disabled before going to sleep. This means that we have the same possible problem as when we added sleeping to the "Superduperloop", if an ISR runs *right after* the scheduler has already selected to go to sleep. The ISR would run, the system would go to sleep, and none of the tasks would get the chance to respond to the ISR until the *next* ISR is run!

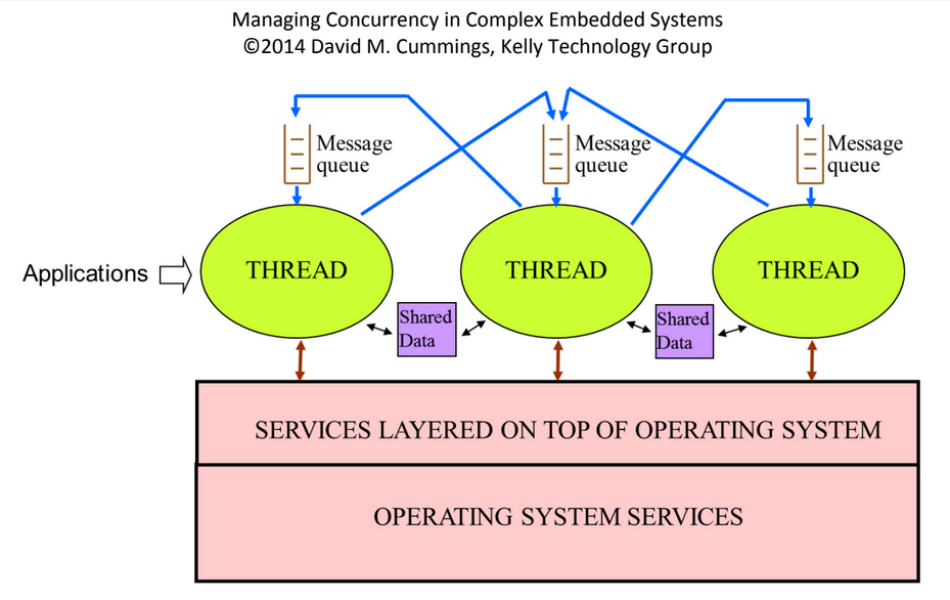
A sample cocoOS application looks like this:

```
static void hello_task(void) {
  task_open();
    for(;;) {
      uart_send_string("Hello World!");
      task_wait( 20 );
    }
  task_close();
}
int main(void) {
  /* Setup ports, clock... */
  system_init();
  /* Create kernel objects */
  task_create( hello_task, 0, 1, NULL, 0, 0 );
  os_init();
  clock_start();
  os_start();
  /* Will never end up here */
  return 0;
}
```

Overall, cocoOS supports a lot of features (the most on this list!) but it suffers for it in terms of complexity. You can find more at the cocoOS GitHub repository.

SST0 (Super-Simple Tasker) and the QV kernel from the QP Framework

SST is build around the paradigm that all tasks are event-driven FSMs, not unlike how we've written our tasks thus far. The biggest difference, however, comes from whether you think of tasks as blocking and waiting for *specific* events (such as thread flags to be set or mutexes to be released) or as waiting for and responding to *any number* of events. In SST and QP, tasks are designed like the latter. The scheduler merely adds events to each task's event queue and calls each task in priority order, with the tasks processing or handling each event in the order in which it was added to its queue. The benefit to this design strategy (as discussed in this paper, from which the graphic below was taken) is that tasks can respond to *any* system event, instead of blocking and waiting for just one event to occur. This becomes important if tasks need to respond to error conditions or other system events that may alter their normal execution.



Messages are sent to other tasks using `SST_Task_post`. Additionally, SST0/QV allows tasks to request that a timer event be posted to their queue after a user-defined timeout value. These timers can be repeated or one-shot. This is a powerful and an incredibly flexible paradigm, one which I like a lot. For *some* applications, though, I don't think this level of flexibility is needed; if you write your tasks in such a way that they essentially reject all messages except the one that they're waiting for, then why have the complexity of a scheduler that allows tasks to respond to any event? But for most applications this is a robust and flexible paradigm and it's the way I imagine writing a lot of my own code.

One downside of this architecture, that I can see, is that it's not really possible for a task to "yield" back to the scheduler (if, for instance, a task needed a long time to process an event but was willing to be interrupted in the middle). Events are consumed when the task runs, so anything that needs to happen with that event has to complete before the task finishes. If that takes a long time, then it might be hard to schedule a set of tasks. One workaround could be for a task to reinsert the event into it's own event queue, forcing the scheduler to run it again, but that feels a *little bit* like a kludge.

One last downside is that SST0 doesn't do anything to recycle the events that get put into each tasks' queues; they are just sort of discarded after they're processed. Each event is a *pointer* to a piece of data, which means discarding this pointer creates a memory leak, unless it's mitigated. The easiest way to mitigate this is by using the heap: tasks that create events use alloc/malloc/calloc to create separate events (with the same data) for each task that they're sending the event to and the scheduler then frees that memory after each task is finished processing the event. If you're averse to using the heap, though, you'll need a more complex solution. Note that the QV kernel *does* include event recycling.

The source code for SST0 can be found here (along with an implementation for a preemptive RTOS, "SST") and, awesomely, it's only 4 source files! I would say that it's a more complicated implementation than ULWOS2, though, in large part because it relies on inheritance to allow tasks to respond to events of different types.

It's my understanding (though I could be wrong) that SST0 is sort of like the minimal, younger brother to the QV (cooperative) kernel from the QP framework. As far as I know, they function the same but the QV kernel is quite a bit more complicated, exactly because the QP framework has a number of advanced features. If you're looking for a robust, cooperative scheduler that's *not* homegrown and is ready to use on *any* commercial product, though, look no further than QV kernel from the QP framework!

Honorable Mention: FreeRTOS

FreeRTOS seems to have some basic support for coroutines (another word describing a cooperative scheduler), and you can even mix coroutines and preemptive tasks, but it's unclear to me if a coroutine can access any part of the preemptive FreeRTOS API (such as to manage mutexes, queues, etc) other than `crDELAY`. Without knowing that, it's hard to recommend this as a viable option for a real system.

DIY Inter-Process Communication

Let's now consider how we might implement these OS primitives ourselves. With only a little bit of work, we'll add a *ton* of capabilities to the "Superduperloop", over which you'll have more control than with any of the projects above.

I like to think of the various forms of IPC as existing in one of four boxes based on whether they are intended as (1) point-to-point versus broadcast signals and whether they are meant to convey either (2) binary data or arbitrary data.

	Point-to-point	**Broadcast**
Binary data	Thread flag/Binary semaphore Counting semaphore	Event flag
Arbitrary data		

	Mailbox Observer pattern Queue Stream buffer Model Points	"Marquee" PubSub

The two techniques that we'll discuss in this article are thread flags/binary semaphores and event flags. (In the next article we'll discuss counting semaphores, mailboxes, queues, a simple Observer pattern, and something I call a "marquee".) Adding these capabilities will only take a few dozen lines of code and will bring the "Superduperloop" up to the level of a full-fledged OS! The required changes will be fairly minimal: to our main scheduler, we'll add a global event flag plus some functions for tasks to register for global events or for a notification when some amount of time has expired.

```
// main.h
//
volatile int g_event_flags;
void registerForEvent(int task_num, int event_flag);
void deregisterFromEvent(int task_num, int event_flag);
void registerForSystickEvent(int task_num, int timeout, int reloadValue);
void deregisterFromSystickEvent(int task_num);
```

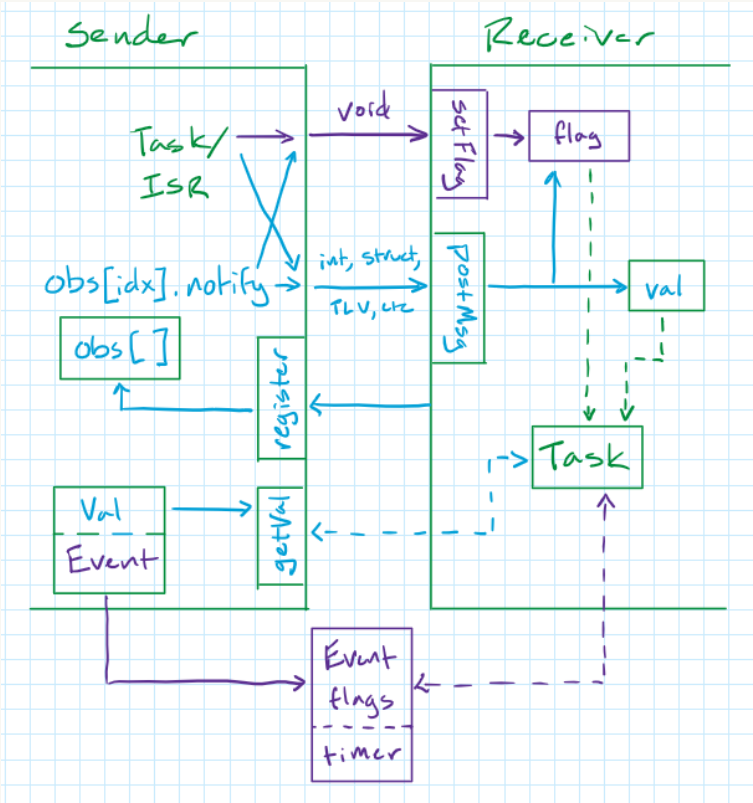
The bigger changes will occur inside each of the tasks, which will gain (1) local variables (to store data or flags sent by other tasks) and (2) public "setter" functions to allow the other tasks to "send" flags or data to that task and "getter" functions to allow other tasks to retrieve information that that task has produced. These public functions are like sockets, allowing other tasks and ISRs that produce events or data to send them to each of our tasks.

```
// TaskA.h
//
/*************************
 *    Prototypes for     *
 *     "Setter" and      *
 *  "Getter" functions   *
 *************************/
// TaskA.c
//
/*************************
 * Local variables/flags *
 *************************/
/*************************
 *     "Setter" and      *
 *  "Getter" functions   *
 *************************/
void TaskA(void){ ... }
```

Additionally, tasks that implement our simple Observer pattern will have a means of registering and deregistering other tasks from receiving notifications when new data is available.

```
// TaskB.h
//
void register( ... );
void deregister( ... );
```

The graphic below depicts how these OS primitives will facilitate data flow to each of the tasks. The two techniques we'll discuss in this article (thread flags/binary sempahores and event flags) are colored in purple; the ones we'll discuss in the next article are colored in light blue.



Each task can alter its execution based on its internal flags or variables (which are set by other tasks or ISRs that call its `"setFlag"` or `"postMsg"` functions) or on global flags or variables; tasks/ISRs implementing the Observer pattern require that a task register itself to be notified when an event occurs.

Okay, let's dive in!

Thread Flags/Binary Semaphores

Thread flags are a point-to-point means of communicating a piece of binary information to a thread: an event did or did not happen, something is or is not true. A semaphore can be used in the same way (among other uses) if it starts at 0 and is given once by an activating event. The `g_tasks` variable in our "Superduperloop" scheduler works exactly like this: many different events can set a task's ready flag to 1, with the task waiting to be run until then. We'll extend this

technique by giving each task it's own flags variable, with one bit associated with each flag or semaphore. ISRs or other tasks can call the appropriate function to both set the flag bit and also set the ready bit in the "Superduperloop"s `g_tasks` variable. Once the task is run, it can check it's own flags variable to see which condition is now true to have caused it to run. The code might look something like this:

```
// TaskA.c
//
/*************************
 * Local variables/flags *
 *************************/
enum { FLAG1_NUM, SEMAPHORE1_NUM };
const int FLAG1_MASK = (1<<FLAG1_NUM);
const int SEMAPHORE1_MASK = (1<<SEMAPHORE1_NUM);
static volatile int thread_flags = 0;
/************************
 *  "Setter" functions   *
 ************************/
void setFlag1(void)
{
    atomic_fetch_or(&thread_flags, FLAG1_MASK);
    atomic_fetch_or(&g_tasks, A_MASK);
}
void giveSemaphore1(void)
{
    atomic_fetch_or(&thread_flags, SEMAPHORE1_MASK);
    atomic_fetch_or(&g_tasks, A_MASK);
}
/************************
 *       The Task        *
 ************************/
void TaskA(void)
{
    int taskStatus = 0;    // 0 = Waiting for event
    static enum { STATE_0, STATE_1 } state = STATE_0;
    switch( state )
    {
        case STATE_0:
            if( thread_flags & FLAG1_MASK )
            {
                // Do stuff when flag1 has been set
                // Reset flag1 atomically
                //
                atomic_fetch_and(&thread_flags, ~FLAG1_MASK);
                // Alt: Clear all pending triggers
                // thread_flags = 0;
                state = STATE_1;
```

```
                taskStatus = 1;          // 1 = Yielded; task should be called
again to complete FSM
            }
            break;
        case STATE_1:
            if( thread_flags & SEMAPHORE1_MASK )
            {
                // Do stuff when semaphore1 has been set
                // Reset semaphore1 and protect critical section
                //
                atomic_fetch_and(thread_flags, ~SEMAPHORE1_MASK);
                // Alt: Clear all pending triggers
                // thread_flags = 0;
                state = STATE_0;
                taskStatus = 0;         // Task is complete; waiting for next
event
            }
            break;
    }
    return taskStatus;
}
```

(Truth be told, a flag bit isn't strictly necessary if the task is able to just check whether or not a condition is true when it runs. However, if there's a concern that the event [such as a very fast button press] might have passed before the task is able to check for it when it runs, or if checking the condition would take a long time, then setting a flag is necessary. I'll assume all events trigger a flag for consistency.)

An ISR or another task can call `setFlag1()` or `giveSemaphore1()` in order to let Task A know that something has happened or that something is now true that wasn't before. Task A starts by waiting for the flag 1 bit to be set, modeling an event that releases the task. If semaphore 1 is given during this state, Task A is still run but it does nothing, effectively ignoring the event. Once Task A has started, it will do some processing and then wait for another event (semaphore 1) before continuing. Again, if the flag 1 bit is set during this state, nothing happens. (It might seem inefficient to run the task even when it will have nothing to do, but I think this same thing is happening in a more complex scheduler anyways. If a semaphore is given by an ISR or task, *something* needs to check if that makes Task A ready to run, whether its the task itself or the scheduler. Putting that check inside the task helps keep our scheduler simple.)

At what point flags get cleared is up to you. If you want each task invocation to start from a clean slate, then you would clear the `thread_flags` variable at the end of each state, before returning from the function. If, on the other hand, you want your task to react to every event, even if it occurred out of order, then you should only clear the individual flag your task is responding to and no others. Consider what would happen in the example above if our events happened out of order, if semaphore 1 was given *before* flag 1. Flag 1 would kick off the task; if you cleared your

flags at the end of the state then Task A would be waiting for a *second* occurrence of semaphore 1 before it could continue. If, on the other hand, you only cleared each flag as it was addressed, then the semaphore 1 flag would still be set when Task A got to State 1. In that case, Task A would run right through it's state machine, *not* needing a second occurrence of semaphore 1 to continue.

We can also use various bitwise operations to easily block until or while some *set* of flags are all true or false, like this:

```
if( flags & 0b11 )        // Blocks until BOTH flag1 and semaphore1 are set
if( flags | 0b11 )        // Blocks until EITHER flag1 or semaphore1 are set;
                          // equivalent, in this case, to "if( flags )"
if( !(flags & 0b11) )     // Blocks WHILE either flag1 or semaphore1 are set;
                          // e.g. UNTIL both flag1 and semaphore1 are CLEARED
```

This gives us considerable flexibility in exactly how a small set of flags or semaphores can trigger a task.

Event Flags

Event flags are a broadcast means of communicating a piece of binary information to many threads: an event did or did not happen, something is or is not true. Whereas a thread flag would be used when only one or a few tasks should know about the event, an event flag would be used if multiple threads needed to know about the event. Task will be allowed to register for (and deregister from) event notifications and just the tasks that have registered for a given event are released when that event occurs (a corresponding bit in a global flag variable is also set to indicate *which* event has just occurred).

```
// main.h
//
enum { EVENT1_FLAG_NUM };
const int EVENT_FLAG1_MASK = (1<<EVENT1_FLAG_NUM);
volatile int g_event_flags = 0;
// main.c
//
static int tasks_registered_for_event1 = 0;
void registerForEvent1(int task_priority)
{
    tasks_registered_for_event1 |= (1<<task_priority);
}
void deregisterFromEvent1(int task)
{
    tasks_registered_for_event1 &= ~(1<<task_priority);
}
void setEventFlag1(void)
{
```

```
    atomic_fetch_or(&g_event_flags, EVENT_FLAG1_MASK);
    atomic_fetch_or(&g_tasks, tasks_registered_for_event1);
}
```

When Event 1 occurs, any task that has previously registered for it will be released. If any of them are waiting on event 1 to occur, they can identify that by reading the `EVENT_FLAG1_MASK` bit in the `g_event_flags` variable. As with thread flags, each task can block on any set of event flags also, using various bitwise operations.

If there are to be a number of event flags in the system, you may find it cleaner to make an array of `tasks_registered_for_event#` with just one register and deregister function to handle all of them.

```
// main.h
//
enum { EVENT1_FLAG_NUM, EVENT2_FLAG_NUM, EVENT3_FLAG_NUM, NUM_EVENT_FLAGS };
const int EVENT_FLAG1_MASK = (1<<EVENT1_FLAG_NUM);
const int EVENT_FLAG2_MASK = (1<<EVENT2_FLAG_NUM);
const int EVENT_FLAG3_MASK = (1<<EVENT3_FLAG_NUM);
volatile int g_event_flags = 0;
// main.c
//
static int tasks_registered_for_events[NUM_EVENT_FLAGS] = {0};
void registerForEvent(int task_num, int event_flag)
{
    tasks_registered_for_events[event_flag] |= (1<<task_num);
}
void deregisterFromEvent(int task_num, int event_flag)
{
    tasks_registered_for_events[event_flag] &= ~(1<<task_num);
}
```

Another thing to consider is when the event flags will be cleared. How and when is entirely up to you as the system designer, but one solution that seemed natural to me is at the bottom of the "Superduperloop" scheduler. Right at the point when the scheduler would go to sleep, we know, with 100% certainty, that each task has had the chance to respond to any event that was just posted. So we'll clear out any event flags there to allow them to be retriggered in the future.

```
if( !g_tasks )
{
    disableInterrupts();
    if( !g_tasks )
    {
        g_event_flags = 0;
        wfi();
    }
```

```
    enableInterrupts();
}
```

One final detail to be addressed is that, given this solution, it would be difficult for tasks to wait for multiple events to all have occurred (e.g. "`if( g_event_flags & (EVT1_MASK | EVT2_MASK) )`"). Since `g_event_flags` gets cleared anytime there's no work to be done, we can reasonably expect it to only have one flag true at a time. For a task to wait on multiple events, it will need to keep track of which events have occurred in its own event flags variable:

```
// main.h
//
enum { EVENT1_FLAG_NUM, EVENT2_FLAG_NUM };
const int EVENT_FLAG1_MASK = (1<<EVENT1_FLAG_NUM);
const int EVENT_FLAG2_MASK = (1<<EVENT2_FLAG_NUM);
volatile int g_event_flags = 0;
// TaskA.c
//
static int my_event_flags = 0;
void TaskA()
{
    ...
        case STATE_0:
            my_event_flags |= g_event_flags;
            if( my_event_flags & (EVENT_FLAG1_MASK|EVENT_FLAG2_MASK) )
            {
                my_event_flags = 0;
                ...
            }
            break;
    ...
}
```

Each time an event occurs, all tasks will be run. Each time Task A is run when it's in State 0, it ORs any current event flags into it's internal variable, `my_event_flags`. Thus, even if the event flag 1 bit is cleared before the event flag 2 bit is set, Task A will remember event flag 1. When the task is finally unblocked, `my_event_flags` is cleared in preparation for the next time the task needs to wait for ALL of a certain number of events to occur.

Timer Events

For events such the timer ISR (that increments `g_systick`), it would be nice if tasks could register for a notification after a *certain period* of time, not necessarily *every* time the event occurs. For instance, maybe a task wants to wait 500 ticks of the system clock; it's a little wasteful to run the task 499 times (once for each tick of the clock) when it doesn't need to run. We can implement this feature by allowing tasks to specify a timeout value (and a reload value)

when they register for a timer event. We'll store this information in an array, with one element for each task.

```
// main.h
//
enum { SYSTICK_POS };
const int SYSTICK_MASK = (1<<SYSTICK_POS);
volatile int g_systick = 0, g_event_flags = 0;
// main.c
//
typedef struct
{
    int timeout;
    int reloadValue;
} timerSettings_t;
static timerSettings_t timerSettings[NUM_TASKS] = {0};
static volatile int timerOverflowBits = 0;
void registerForSystickEvent(int task_num, int timeout, int reloadValue)
{
    timerSettings[task_num].timeout = timeout;
    timerSettings[task_num].reloadValue = reloadValue;
    resetOverflowBit(task_num);
}
void deregisterFromSystickEvent(int task_num)
{
    timerSettings[task_num].timeout = 0;
}
```

Every time the timer ISR runs it loops through the array of timer settings, decrementing the timeout values of any timers that are currently running (i.e. whose `timeout` values are greater than 0). If the `timeout` value *was* greater than 0 and then *is* 0 after it's been decremented, then the task is released, a corresponding overflow bit is set, and the `timeout` value is reset to the value in `reloadValue`.

```
void timerISR(void)
{
    g_systick++;
    atomic_fetch_or(&g_event_flags, SYSTICK_MASK);
    for( size_t idx = 0; idx < NUM_TASKS; idx++ )
    {
        if( timerSettings[idx].timeout > 0 )
        {
            if( --timerSettings[idx].timeout == 0 )
            {
                atomic_fetch_or(&g_tasks, (1<<idx));
                timerOverflowBits |= (1<<idx);
                timerSettings[idx].timeout = timerSettings[idx].reloadValue;
```

```
            }
        }
    }
}
```

A task that is waiting on a timer can then check if it's overflow bit is set to identify when it runs that it was released by the system timer.

```
// main.c
bool timerOverflowed(int task_num)
{
    return (timerOverflowBits & (1<<task_num));
}
void resetOverflowBit(int task_num)
{
    atomic_fetch_and(&timerOverflowBits, ~(1<<task_num));
}
// TaskA.c
void TaskA(void)
{
    ...
    registerForSystickEvent( A_NUM, 500, 0 );   // Set up a one-shot timer that
will
                                                // expire after 500 system ticks
    if( timerOverflowed( A_NUM ) )
    {
        // Do something when the timer has expired
        resetOverflowBit( A_NUM );
    }
    ...
}
```

"Anything you can do, I can do better!"

A neat feature of the way we've designed our "Superduperloop" is that a task can wait until *any arbitrary C expression* is true before continuing to run. We're not limited to just the mechanisms our RTOS provides (most of which only allow a task to wait for a *single* event, with an optional timeout). That means we can write code that blocks until any one of *several* conditions are true, such as:

```
if( (thread_flags & SEMAPHORE1_MASK) || timerOverflowed( A_NUM ) || spiNack() )
```

Additionally, we can run code every time our task runs, even if it's still blocked. We saw an example of this, above, when we ORed `g_event_flags` with a tasks's `my_event_flags` to be able to block until multiple events had all occurred. Another example might be the situation where we

want to know how many times a task is called before it gets to run. We can code that with something like the following:

```
static task_count = 0;
void TaskA()
{
    ...
        case STATE_0:
            task_count++;
            if( thread_flags & SEMAPHORE1_MASK )
            {
                // Do something with task_count
                task_count = 0;
    ...
}
```

In fact, if you're using GCC, then the conditional expression unblocking our task can also be a *statement expression*, combining the last two techniques:

```
static task_count = 0;
void TaskA()
{
    ...
        case STATE_0:
            if( ({ task_count++; thread_flags & SEMAPHORE1_MASK;}) )
            {
                // Do something with task_count
                task_count = 0;
    ...
}
```

The possibilities abound. Let's see an RTOS do any of *that*!

The "Superduperloop", v2

This, then, is the "Superduperloop", *version 2*, with added support for thread flags/binary semaphores and event flags.

```
/*****************************************
*     main.h                             *
*****************************************/
// Task priorities/bitmasks, one per task
// E.g. enum { A_NUM, NUM_TASKS };
//      const int A_MASK = (1<<A_NUM);
//
// Event flags
```

```
// E.g. enum { SYSTICK_NUM, EVENT_FLAG1_NUM, EVENT_FLAG1_NUM, NUM_EVENT_FLAGS };
//      const int SYSTICK_MASK = (1<<SYSTICK_NUM);
//      const int EVENT_FLAG1_MASK = (1<<EVENT_FLAG1_NUM);
//      const int EVENT_FLAG2_MASK = (1<<EVENT_FLAG1_NUM);
//
// Global variables
volatile int g_systick = 0, g_tasks = 0, g_event_flags = 0;
//
/***************************************
*     main.c                           *
***************************************/
// Variables to store which tasks have registered for each event
static int tasks_registered_for_events[NUM_EVENT_FLAGS] = {0};
//
// Typedef and array to hold timer information for each task
typedef struct
{
    int timeout;
    int reloadValue;
} timerSettings_t;
static timerSettings_t timerSettings[NUM_TASKS] = {0};
static volatile int timerOverflowBits = 0;
//
// Function prototypes for tasks here
// E.g.  int TaskA(void);
//
//-----Event (de)registration functions------//
void registerForEvent(int task_num, int event_flag)
{
    tasks_registered_for_events[event_flag] |= (1<<task_num);
}
void deregisterFromEvent(int task_num, int event_flag)
{
    tasks_registered_for_events[event_flag] &= ~(1<<task_num);
}
//--End of Event (de)registration functions--//
//--------------Timer functions--------------//
void registerForSystickEvent(int task_num, int timeout, int reloadValue)
{
    timerSettings[task_num].timeout = timeout;
    timerSettings[task_num].reloadValue = reloadValue;
    resetOverflowBit(task_num);
}
void deregisterFromSystickEvent(int task_num)
{
    timerSettings[task_num].timeout = 0;
}
bool timerOverflowed(int task_num)
```

```
{
    return (timerOverflowBits & (1<<task_num));
}
void resetOverflowBit(int task_num)
{
    atomic_fetch_and(&timerOverflowBits, ~(1<<task_num));
}
void timerISR(void)
{
    g_systick++;
    atomic_fetch_or(&g_event_flags, SYSTICK_MASK);
    for( size_t idx = 0; idx < NUM_TASKS; idx++ )
    {
        if( timerSettings[idx].timeout > 0 )
        {
            if( --timerSettings[idx].timeout == 0 )
            {
                atomic_fetch_or(&g_tasks, (1<<idx));
                timerOverflowBits |= (1<<idx);
                timerSettings[idx].timeout = timerSettings[idx].reloadValue;
            }
        }
    }
}
//-----------End of Timer functions----------//
//--------------------Task------------------//
void main(void)
{
    while(1)
    {
        // One if/else if block per task, higher priority tasks located
        // higher up in the chain of if statements.
        //
        if( g_tasks & A_MASK )
        {
            atomic_fetch_and(&g_tasks, (~A_MASK));
            if( TaskA() ) atomic_fetch_or(&g_tasks, A_MASK);
        }
        else if( g_tasks & B_MASK )
        {
            atomic_fetch_and(&g_tasks, (~B_MASK));
            if( TaskB() ) atomic_fetch_or(&g_tasks, B_MASK);
        }
        else
        {
            if( !g_tasks )
            {
                disableInterrupts();
```

```
                if( !g_tasks )
                {
                    g_event_flags = 0;
                    wfi();
                }
                enableInterrupts();
            }
        }
    }
}
```

The biggest change to our main scheduler was to add a set of functions related to the system clock that allows tasks to register for timer notifications when a time value they specify expires. The other changes to the main scheduler were pretty minimal:

1. we moved the task priorities enumeration, a few constant ints, and the variables `g_systick`, `g_tasks`, and `g_event_flags` to `main.h` so that the tasks could reference them,
2. we added an enumeration, a few constant ints, and an array for the event flags, and
3. we added one line of code to reset the event flags right as the system would go to sleep.

That's it! The tasks were outfitted with local flags and with public "setter" functions to allow other tasks to set those flags. A sample task that demonstrates these two OS primitives is shown below. I've chosen to use the Protothreads library to help keep my FSM syntax succinct (which would also change the signature for the function prototypes in the associated `main.c` file).

```
// Receiver.c
//
//------------------Thread flags-------------------------//
enum { FLAG1_NUM, SEM1_NUM, NUM_FLAGS_TASK_A };
const int FLAG1_MASK = (1<<FLAG1_NUM);
const int SEM1_MASK = (1<<SEM1_NUM);
static int thread_flags = 0;
//--------------"Setter" functions-----------------------//
void setFlag1(void)
{
    atomic_fetch_or(&thread_flags, FLAG1_MASK);
    atomic_fetch_or(&g_tasks, A_MASK);
}
void giveSem1(void)
{
    atomic_fetch_or(&thread_flags, SEM1_MASK);
    atomic_fetch_or(&g_tasks, A_MASK);
}
//-----------------------Task---------------------------//
int TaskA(struct pt *pt)
{
    float temp;
```

```
    PT_BEGIN(pt);
    PT_WAIT_UNTIL( pt, thread_flags & FLAG1_MASK );
    // Do stuff when flag1 has been set
    thread_flags = 0;
    registerForEvent( A_NUM, EVENT_FLAG2_NUM );
    registerForSystickEvent( A_NUM, 500, 0 );      // Create a one-shot timer to
expire after 500 system ticks
    PT_WAIT_UNTIL( pt, (event_flags & EVENT_FLAG1_MASK) ||
timerOverflowed( A_NUM ) );
    deregisterFromEvent( A_NUM, EVENT_FLAG2_NUM );
    if( event_flags & EVENT_FLAG1_MASK )
    {
        // Do something after event flag 2 has been set, otherwise...
    }
    else
    {
        // ...handle timeout
        resetOverflowBit( A_NUM );
    }
    PT_WAIT_UNTIL( pt, thread_flags & SEM1_MASK );
    // Do stuff when semaphore1 has been given
    PT_RESTART(pt);    // Resume on next task release at "PT_BEGIN"
    PT_END(pt);
}
```

Summary

Real embedded applications need not only a scheduler but also a means for the tasks in that application to communicate with each other (inter-process communication [IPC]). We can organize the various forms of IPC by whether they are meant (1) to be point-to-point or broadcast signals and (2) to convey binary or arbitrary data.

	Point-to-point	**Broadcast**
Binary data	Thread flag/Binary Semaphore Counting semaphore	Event flag
Arbitrary data	Mailbox Observer Message queue Stream buffer Model Points	Marquee PubSub

The best "off the shelf" options for fixed-priority, non-preemptive schedulers include ULWOS2, cocoOS, and SST0/QV. Additionally, with a little bit of work, it's not hard to add all of the above

forms of IPC to the "Superduperloop" (of which we discussed two in this article). The tables below summarize these options; each solution offers a different mix of features and complexity.

Advantages and Disadvantages

	Advantages	Disadvantages
ULWOS2	• Simple	• Employs something akin to a dynamic task priority scheme for tasks assigned the same priority level, complicating schedulability analysis • No support for sleeping if idle • Uses `goto`s to implement coroutine behavior (instead of `switch...case`), which may be undesirable for some
cocoOS	• The most features of any OS here	• Supports sleeping, but with possible error • All messages inherit from a base class (use of inheritance [especially in C!] may be difficult for some)
SST0/QV	• Highly reputable author (Miro Samek) • Likely to be the most stable/bug-free	• Uses message-passing paradigm only (may feel odd to some) • All events inherit from a base class (use of inheritance [especially in C!] may be difficult for some) • SST0 is limited (as written) to only 8 tasks • Not (easily) possible for tasks to yield in the middle of processing an event • SST0 does not currently recycling the memory used to create events (possible memory leak)
"Superduperloop"	• Simple • Tasks can wait on any valid C/C++ expression	• (Currently) Only supports thread and event flags • It may feel tedious (to some) to need to write separate functions inside each

		task's source file for setting the individual thread flags

Features

	ULWOS2	cocoOS	SST0/QV	"Superduperloop"
Thread flags	N	N	N	Y
Binary Semaphores	N	Y (but not thread-safe)	N	Y
Event flags	Y	Y	N (but does include timer notifications)	Y
Mailbox/ Message queues	Y (but not thread-safe)	Y (but not thread-safe)	Y	N
Complexity	Medium (~500 LOC)	High (~2000 LOC)	• SST0: Medium (~400 LOC) • QV: High (~1900 LOC)	Low (~50-100 LOC)

Hopefully now you're ready to start writing your next embedded application with one of those schedulers, absent (almost) any concerns about race conditions! In the last article in this series we'll design our third (and final) iteration of the "Superduperloop", that will include support for not just thread and event flags but also counting semaphores, mailboxes, message queues, the Observer pattern, and "marquee"s. Additionally, I'll show you how you can further improve the schedulability of your task sets by using a *dynamic*-priority, non-preemptive scheduler designed around a **dispatch queue** (i.e. a queue of tasks). Stay tuned, my friends, for our thrilling series finale!

References

- Other cool Superloops
  - Protothreads
  - ULWOS2 (article: part 1 and part 2 | code)
  - cocoOS
  - SST0 (video | code) / QV (docs | code)
- Documentation about OS primitives / IPC
  - FreeRTOS

    - CMSIS RTX5
- Managing Concurrency in Complex Embedded Systems by David Cummings
- Messages for Intertask Communication by D. Kalinsky
- Event Queue by Game Programming Patterns